

# BÁO CÁO TUẦN 1 — Thu thập dữ liệu và Pipeline nhúng

Thời gian: 29/05 – 05/06/2026 • Trạng thái: Đã hoàn thành

*Trợ lý hồi đáp thủ tục hành chính công Việt Nam ứng dụng Agentic RAG*

Nhóm: Nguyễn Xuân Phúc, Nguyễn Minh Triết

GVHD: Đỗ Trọng Hợp, Nguyễn Ngọc Quý

## 1 Mục tiêu tuần

Mục tiêu của tuần đầu tiên là xây dựng hoàn chỉnh một pipeline dữ liệu đầu cuối (end-to-end), bao gồm các khâu: thu thập toàn bộ thủ tục hành chính công từ Cổng Dịch vụ công Quốc gia, chuẩn hoá dữ liệu thô thành một corpus có cấu trúc, nhúng (embedding) và nạp (ingest) vào cơ sở dữ liệu vector. Đây là nền tảng tri thức cho toàn bộ hệ thống RAG được phát triển ở các tuần kế tiếp.

## 2 Công việc đã thực hiện

**Phân tích nguồn và dịch ngược API (Reverse-engineering):** Cổng Dịch vụ công Quốc gia<sup>1</sup> được xây dựng theo kiến trúc ứng dụng trang đơn (Single-Page Application), trong đó nội dung thủ tục được nạp động qua một lớp REST API nội bộ không công bố tài liệu. Vì lý do này, các kỹ thuật cào dữ liệu dựa trên phân tích cây HTML không khả dụng. Nhóm tiến hành phân tích lưu lượng mạng (network traffic analysis) để khôi phục đặc tả của hai endpoint công khai cần thiết (mang hậu tố `by-citizen`, thuộc route `/api/v1`): một endpoint liệt kê danh mục và một endpoint lấy chi tiết. Cả hai đều không yêu cầu xác thực và trả về dữ liệu JSON theo cấu trúc chuẩn hoá (mã trạng thái kèm trường dữ liệu).

**Xây dựng crawler hai giai đoạn:** Quy trình thu thập được tổ chức tuần tự qua hai giai đoạn. Giai đoạn duyệt danh mục (Listing) quét toàn bộ danh sách thủ tục bằng cơ chế phân trang con trỏ (cursor-based pagination): mỗi request gửi kèm định danh phần tử cuối của trang trước (tham số `lastId`) làm mốc để máy chủ trả về lô tiếp theo, vòng lặp dừng khi nhận được danh sách rỗng. So với phân trang theo chỉ số trang, cơ chế này giảm thiểu rủi ro trùng lặp hoặc bỏ sót khi danh mục thay đổi ngay trong lúc thu thập. Kết quả là một bộ chỉ mục rút gọn (định danh, mã thủ tục, tên, lĩnh vực, cơ quan) cho từng thủ tục. Giai đoạn thu thập chi tiết (Detailing) lần lượt gọi endpoint thứ hai theo từng định danh để tải bản ghi đầy đủ, lưu thành một tệp JSON độc lập cho mỗi thủ tục tại `data/raw/`.

**Cơ chế chịu lỗi và tiết chế tải (Fault Tolerance & Rate Limiting):** Do quy mô lên tới hàng nghìn request, crawler được thiết kế để chịu lỗi và tránh gây quá tải cho máy chủ nguồn. Giữa các request, hệ thống chèn một khoảng trễ cơ sở kết hợp nhiễu ngẫu nhiên (random jitter) để dàn đều lưu lượng. Khi gặp lỗi mạng hoặc lỗi phía máy chủ, request được thử lại với thời gian chờ tăng lũy thừa (exponential backoff). Mỗi bản ghi được ghi xuống đĩa theo cơ chế atomic write (xuất ra tệp tạm rồi đổi tên) để tránh sinh ra tệp JSON hỏng khi tiến trình bị ngắt. Trạng thái phân trang và tập định danh đã tải được lưu lại (checkpointing), cho phép tạm dừng và tiếp tục (resume) mà không xử lý lại từ đầu; các định danh thất bại được ghi vào một tệp nhật ký riêng để thử lại sau.

<sup>1</sup><https://dichvucong.gov.vn>



Figure 1: Quá trình thu thập dữ liệu từ Cổng Dịch vụ công Quốc gia.

**Chuẩn hoá và phân đoạn (Parsing & Chunking):** Mỗi bản ghi JSON thô được chuyển thành một tài liệu Markdown độc lập (self-contained), gồm hai phần: một khối metadata front-matter (mã thủ tục, lĩnh vực, cấp thực hiện, cơ quan, đối tượng) phục vụ lọc và truy vết, cùng phần thân chứa sáu hạng mục đã làm sạch (trình tự, cách thức, thành phần hồ sơ, điều kiện, căn cứ pháp lý, kết quả). Bước này xử lý nhiều đặc thù của dữ liệu nguồn: ánh xạ các giá trị enum sang tiếng Việt (hình thức nộp, đơn vị thời hạn, cấp thực hiện), chuẩn hoá mức phí (giá trị 0 thành Miễn phí), và đặc biệt là bóc tách thành phần hồ sơ vốn nằm lồng bên trong mảng `executionCases` thay vì ở cấp gốc. Thay vì cắt văn bản theo độ dài cố định, nhóm áp dụng phân đoạn theo cấu trúc ngữ nghĩa (section-based chunking): mỗi hạng mục là một chunk có ngữ cảnh trọn vẹn, gắn tiền tố định danh `[Tên thủ tục - Tên mục]` ở đầu. Đầu ra gồm `chunks.jsonl` (mỗi dòng một chunk kèm metadata) và `metadata.jsonl`.

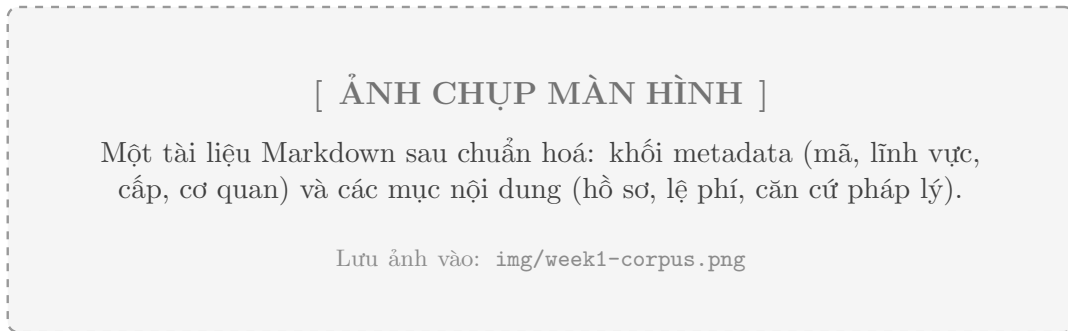


Figure 2: Tài liệu corpus sau bước chuẩn hoá và phân đoạn theo mục.

**Nhúng và lưu trữ (Embedding & Storage):** Mỗi chunk được biểu diễn thành một vector 3072 chiều thông qua mô hình `text-embedding-3-large`. Để tối ưu thời gian xử lý corpus lớn, quá trình nhúng được thiết kế chạy song song đa luồng (multi-threading) với cơ chế gộp lô (batching) và thử lại theo exponential backoff để đối phó với giới hạn rate-limit của API, rút thời gian từ khoảng 2,5 giờ xuống còn khoảng 15 phút. Kho tri thức đầu ra gồm ba lớp lưu trữ đồng bộ (Hình 3): (a) chỉ mục vector dựa trên đồ thị HNSW tối ưu cho tìm kiếm ngữ nghĩa; (b) chỉ mục toàn văn bản (full-text index) để tra cứu chính xác các thực thể cứng như mã biểu mẫu (CC01) hay số hiệu văn bản (Thông tư 17/2024/TT-BCA); và (c) một catalog cơ sở dữ liệu quan hệ ánh xạ Tài liệu, Chunk và Vector phục vụ truy vết nguồn. Sau khi nạp, hệ thống chạy kiểm tra toàn vẹn (integrity check) để đối chiếu số lượng nguồn, đoạn và vector giữa ba lớp, đồng thời xác minh các tệp nhị phân của HNSW; cuối cùng tạo full-text index và đóng gói toàn bộ index để chia sẻ qua HuggingFace nhằm tái lập trên máy khác.

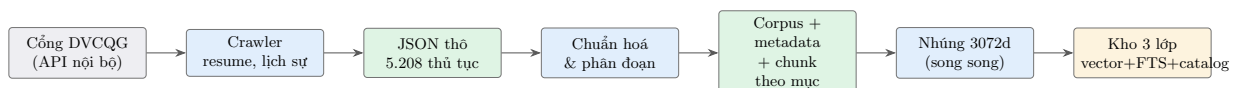


Figure 3: Pipeline dữ liệu đầu cuối của tuần 1.

## [ ẢNH CHỤP MÀN HÌNH ]

Log nhúng song song nhiều luồng và kiểm tra toàn vẹn (số nguồn, đoạn, vector khớp nhau giữa ba kho); hoặc thư mục index đã dựng.

Lưu ảnh vào: `img/week1-ingest.png`

Figure 4: Nhúng corpus và kiểm tra toàn vẹn kho tri thức ba lớp.

### 3 Kết quả và sản phẩm bàn giao

| Thành phần                            | Mô tả                                                              |
|---------------------------------------|--------------------------------------------------------------------|
| <code>pipeline/crawler/</code>        | Mã crawler và cấu hình (endpoint, độ trễ, backoff).                |
| <code>pipeline/parser/parse.py</code> | Bộ chuẩn hoá JSON sang Markdown và sinh chunk.                     |
| <code>data/raw/*.json</code>          | 5.208 bản ghi thô.                                                 |
| <code>data/corpus/md/*.md</code>      | 5.208 tài liệu Markdown kèm metadata.                              |
| <code>data/corpus/chunks.jsonl</code> | Chunk theo mục kèm metadata.                                       |
| <code>rag/fast_ingest.py</code>       | Nhúng song song vào Chroma, LanceDB và SQLite.                     |
| Index ( <code>ktem_data</code> )      | Vectorstore, docstore và catalog đã dựng, chia sẻ qua HuggingFace. |

| Chỉ số          | Giá trị        | Chỉ số                | Giá trị |
|-----------------|----------------|-----------------------|---------|
| Số thủ tục      | 5.208          | Số chiều vector       | 3072    |
| Số luồng nhúng  | 10             | Kích thước lô (batch) | 16      |
| Thời gian nhúng | khoảng 15 phút | Số lớp lưu trữ        | 3       |

### 4 Khó khăn và hướng xử lý

- API không có tài liệu công khai: nhóm khôi phục đặc tả bằng phân tích lưu lượng mạng và kiểm chứng (verify) trên một tập nhỏ trước khi thu thập toàn bộ.
- Thành phần hồ sơ không nằm ở cấp gốc: trường dữ liệu này lồng bên trong `executionCases`, nhóm xác định đúng vị trí lồng nhau để bóc tách đầy đủ giấy tờ.
- Đường dẫn chứa ký tự Unicode trên Windows làm thư viện HNSW không ghi được tệp nhị phân (mất vector âm thầm): khắc phục bằng cách ép biến `KH_APP_DATA_DIR` trở tới một đường dẫn ASCII.
- Treo cache khi tiến trình nhúng bị ngắt: bản nhúng song song bỏ qua lớp cache trung gian nên an toàn khi dừng và chạy lại.

### 5 Checklist tuần 1

- ☑ Dịch ngược API và xây crawler hai giai đoạn (resume, backoff), thu thập 5.208 bản ghi.
- ☑ Parser chuẩn hoá JSON sang Markdown kèm metadata và phân đoạn theo mục.
- ☑ Nhúng Azure 3072 chiều song song và dựng kho lưu trữ ba lớp.
- ☑ Kiểm tra toàn vẹn, tạo full-text index và đóng gói chia sẻ index.

Hướng tuần kế tiếp: khảo sát các phương pháp RAG và triển khai đường cơ sở (baseline Simple) có truy hồi lai ghép và trích dẫn trên giao diện.